



[BIT703 Web Technologies](#) / [Contents](#) / [Topic 12: Web application performance](#) / [Testing techniques](#)

White-box techniques 45m



© © Getty Images



[Skip to main content](#)

White-box testing is a method in which testers evaluate the internal structure, design and coding of the software under test (SUT). This technique is sometimes called clear-box, glass-box, open-box, transparent-box or structural testing – terms that highlight the technique's aim to see through the software's box to its inner workings (structure-based techniques).

White-box testing ensures that all internal operations are performed according to the specifications.

White-box testing includes testing the following:

- internal security
- broken paths in the code
- the flow of inputs through the code
- expected outputs
- functionality of loops
- each statement, object or function.

This testing can be done at different test levels, from unit test levels to integration and system test levels. It can be time-consuming and expensive, depending on the size of the AUT or SUT. It also requires an understanding of coding and implementation, so is often automated and executed by developers.

There are various white-box testing tools available. Some examples are:

- Parasoft Jtest
- Fiddler
- OpenGrok
- sqlmap.

In terms of program code, this also includes code coverage.

Code coverage analysis

Code coverage analysis is a major white-box testing technique. This aims to identify areas of a program that are not exercised by test cases, then creates test cases to verify these untested parts of the code. Two code coverage analysis techniques are discussed here.

[Skip to main content](#)

Statement coverage

Statement coverage is a method of validating whether each line of the code is executed at least once. It can be used to check the flow of different paths in the program but can't be used to check a false condition. Statement coverage can be calculated using the equation below and is stated as a percentage.

$$\text{statement coverage} = \left(\frac{\text{number of statements executed}}{\text{total number of statements in the source code}} \right) \times 100\%$$

For example:

- Input values = X, Y
- Input result is $Z = X + Y$
- If $Z > 10$
- Print "Z is greater than 10"

If your test case is $X = 5$, $Y = 7$, you have tested all four statements (or lines of code), which is 100% statement coverage.

Decision coverage (branch coverage)

This technique is used to validate whether each branch of code is executed at least once.

Using the above example and extending it:

- Input values = X, Y
- Input result is $Z = X + Y$
- If $Z > 10$
- Print "Z is greater than 10"
- Else
- Print "Z is less than 10"

[Skip to main content](#)

If your test case is $X = 5, Y = 7$, you're only testing four out of six statements (lines of code), which is 66% coverage.

If you add a second test case of $X = 3, Y = 4$, you're testing all six statements (lines of code), giving you 100% coverage.

So, two test cases are required to execute both decision branches.

Statement coverage and decision coverage enable:

- validation of the flow of specific input through the code
- detection of unused paths in the code
- detection of internal security holes
- validation of loops/decisions at their boundaries
- discovery of logical errors and design errors
- discovery of syntax errors and typographical errors.

A good, well covered example of this type of testing in action can be found with the most ubiquitous database on the planet, SQLite. This project has 590 times more test code than there is program code that makes up the actual SQLite database. You can say they have tested almost every possible thing that they can test within the code base and have left nothing untested.

To summarise, they have:

- four independently developed test harnesses
- 100% branch test coverage in an as-deployed configuration
- millions and millions of test cases
- out-of-memory tests
- I/O error tests
- crash and power loss tests
- fuzz tests
- boundary value tests
- disabled optimisation tests
- regression tests
- malformed database tests

■ Skip to main content of assert() and run-time checks

- Valgrind analysis
- undefined behaviour checks
- checklists

and more!



☒ White-box testing

Non-Assessed

Using the PHP code below, calculate the code coverage (statement coverage) if $a = 4$ and $b = 4$.

```
<?php
if ($a > $b) {
    echo "a is bigger than b";
} elseif ($a == $b) {
    echo "a is equal to b";
} else {
    echo "a is smaller than b";
}
?>
```

☐ I have completed the task

Submit



[Skip to main content](#)

Thoughts on this page?

